

Comprehensive Guide

Cross-Machine QUIC Server Migration: A Comprehensive Guide

Table of Contents

1. Background: What is QUIC?
 2. The Problem: Why Server Migration?
 3. How QUIC Migration Normally Works
 4. What We Built: Cross-Machine Migration
 5. The Architecture: Four Machines
 6. Step-by-Step: What Happens During Migration
 7. The Key Challenge: Moving Encryption Keys
 8. Code Changes to Neqo
 9. The Migration State Format
 10. How the Preferred Server Handles Packets
 11. When to Transfer State (Timing Strategies)
 12. How to Transfer State (Mechanisms)
 13. Combining When and How
 14. Modular Implementation
 15. Benchmark Results
 16. Multi-Server Migration (N:M)
 17. Security Implications
 18. Running the Demo
 19. Future Work
 20. Glossary
-

1. Background: What is QUIC?

QUIC is a modern transport protocol developed by Google and standardized by the IETF in RFC 9000. It runs on top of UDP and is used by HTTP/3 – the latest version of the web protocol. Think of it as a faster, more secure replacement for TCP+TLS.

Why QUIC matters

- **Handshake time:** TCP+TLS needs 2-3 round trips; QUIC needs 1 round trip (or 0 with 0-RTT)
- **Encryption:** TCP+TLS is optional (TLS is a separate layer); QUIC is always encrypted, built-in
- **Connection identity:** TCP is tied to IP address + port; QUIC is tied to Connection IDs
- **Head-of-line blocking:** TCP yes (one lost packet blocks everything); QUIC no (streams are independent)

Connection IDs: The Key Concept

In TCP, a connection is identified by a 4-tuple: (source IP, source port, destination IP, destination port). If any of these change (e.g., you switch from Wi-Fi to cellular), the connection breaks.

QUIC solves this with **Connection IDs (CIDs)**. Each side of a QUIC connection picks one or more random identifiers. Packets include these CIDs in their headers. As long as the CID matches, the connection stays alive – even if the IP address changes.

```
TCP connection identity:    192.168.1.5:12345 <-> 10.0.0.1:443
QUIC connection identity:  CID = 0x7a3f9b2c (doesn't depend on IP)
```

How QUIC Encryption Works

Every QUIC packet (except the very first ones) is encrypted. The encryption uses:

1. **A TLS 1.3 handshake** – Client and server agree on a shared secret
2. **Key derivation** – From the shared secret, both sides derive:
 - An **AEAD key** (for encrypting/decrypting packet payloads)
 - A **Header Protection (HP) key** (for masking packet numbers in headers)
3. **Separate keys for each direction** – The server's "write key" is the client's "read key" and vice versa

```
Shared TLS Secret
|
+--> Server Write Key (= Client Read Key)
|    +-- AEAD key (encrypts payload)
|    +-- HP key (masks packet number)
|
+--> Server Read Key (= Client Write Key)
|    +-- AEAD key (decrypts payload)
|    +-- HP key (unmasks packet number)
```

2. The Problem: Why Server Migration?

Server-Side Migration (RFC 9000, Section 9.6)

QUIC includes a feature called **Preferred Address**. During the handshake, the server can tell the client:

"Hey, I'm at 10.0.0.1:443 right now, but I'd prefer you talk to me at 10.0.0.2:443 instead."

The client then migrates the connection to the new address. This is useful for:

- **Load balancing** – Move clients from an overloaded server to a less busy one
- **Geographic optimization** – Redirect to a closer server after initial contact
- **Server maintenance** – Drain connections before shutting down a server

The Limitation in Existing Implementations

Every QUIC implementation (including Mozilla's Neqo, which we used) assumes that the preferred address is **on the same machine**. It's just a different network interface or IP address on the same box.

The core problem: How do you move the encryption keys from Machine A to Machine B so that Machine B can decrypt the client's packets and respond?

3. How QUIC Migration Normally Works

Here's the standard preferred address migration flow:

```
Client                                Primary Server
|                                     |
| -- Initial (ClientHello) --> |
| <-- Initial (ServerHello) --| Server includes "preferred_address"
|   + Handshake                |
|   + preferred address:       |
|     10.0.0.2:4433           |
| -- Handshake Complete ----> |
| -- PATH CHALLENGE -----> 10.0.0.2:4433
|   (random 8-byte token)      (preferred addr)
|                               |
| <-- PATH RESPONSE ----- 10.0.0.2:4433
|   (same 8-byte token)        (proves it's the same server)
|
| Path validated! Client now sends all traffic to 10.0.0.2:4433
```

PATH_CHALLENGE / PATH_RESPONSE

These are special QUIC frames used for **path validation**:

1. Client generates a random 8-byte value
2. Client sends it in a PATH_CHALLENGE frame to the preferred address
3. The preferred address must respond with the exact same 8 bytes in a PATH_RESPONSE frame
4. Since the response is encrypted with the connection's keys, only the real server can produce it

This is critical: **the server at the preferred address must have the same encryption keys** to decrypt the challenge and encrypt the response.

4. What We Built: Cross-Machine Migration

We modified Mozilla's Neqo QUIC implementation to support migration across **physically separate machines**. This required solving a problem that no existing QUIC implementation handles: transferring the cryptographic state from one machine to another.

What is Neqo?

Neqo is Mozilla's QUIC implementation, written in Rust. It's used in Firefox for HTTP/3 connections. We chose it because:

- It's open source and well-structured

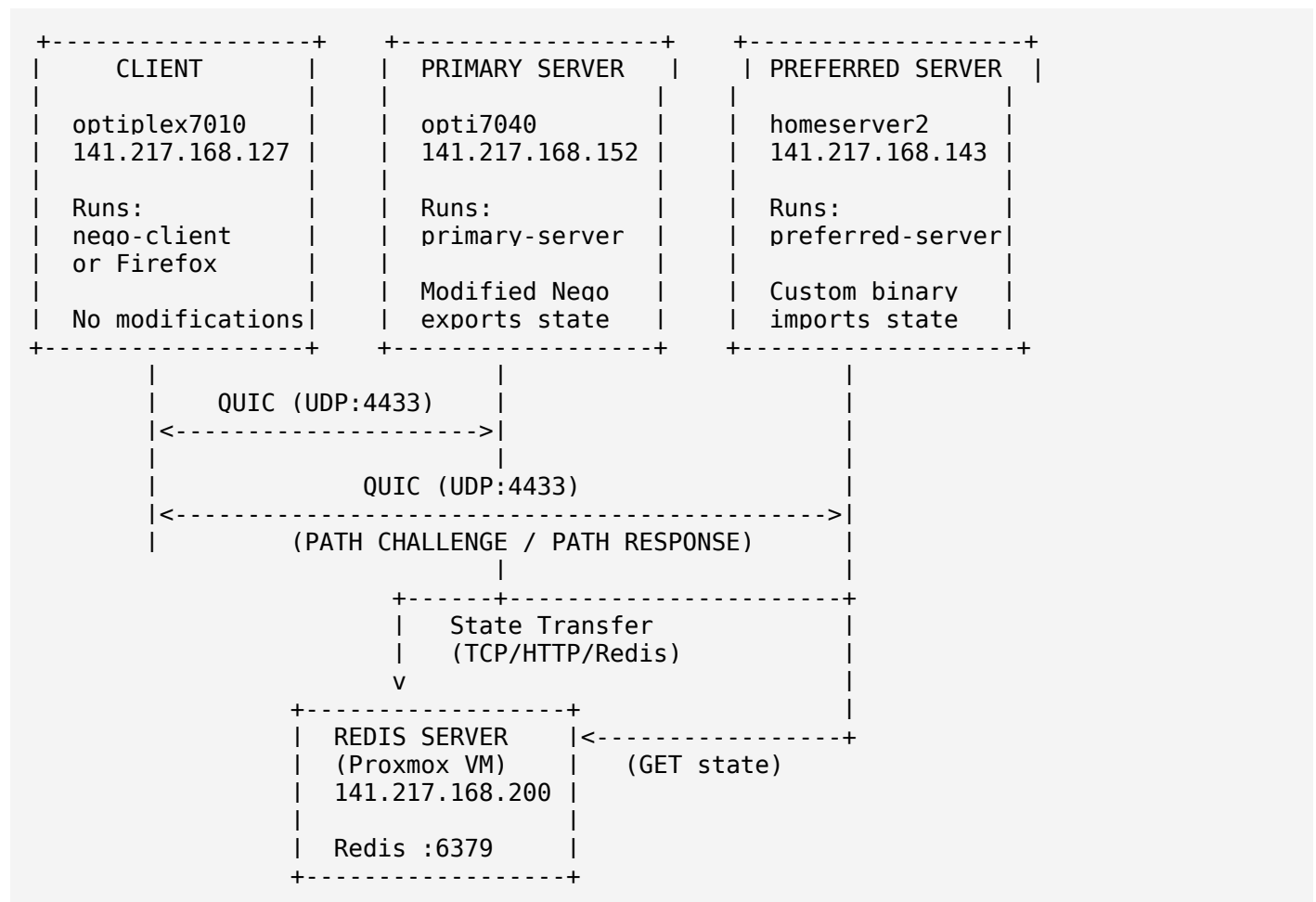
- Written in Rust (memory-safe, good for security research)
- Uses Mozilla’s NSS library for cryptography (the same library Firefox uses)
- Has clear separation between crypto, transport, and connection logic

What We Changed (High-Level)

1. **Made the crypto keys exportable** – Neco normally discards the raw TLS secrets after deriving encryption keys. We modified it to retain and expose them.
2. **Created a serialization format** – We defined a binary format to pack all the connection state (keys, connection IDs, client address, etc.) into bytes that can be sent over the network.
3. **Built import logic** – We wrote code that takes those bytes and reconstructs working encryption/decryption objects on the receiving server.
4. **Wrote two new programs** – A “primary server” that exports state, and a “preferred server” that imports it and handles PATH_CHALLENGE.
5. **Built 4 pluggable state transfer backends** – TCP Push, HTTP Pull, Redis KV, Redis Pub/Sub, selectable via environment variable.

5. The Architecture: Four Machines

Our demo uses four physical machines on the same local network:



Why Four Machines?

- The **client** is unmodified – it's a standard QUIC client (or Firefox browser). It doesn't know that multiple servers are involved.
- The **primary server** handles the initial connection and handshake, then hands off.
- The **preferred server** takes over the connection using the exported crypto state.
- The **Redis server** runs on a dedicated Proxmox VM, keeping the state transfer infrastructure independent from both QUIC servers.

This separation proves that the migration is truly cross-machine, not just within one process.

6. Step-by-Step: What Happens During Migration

Phase 0: Startup

1. Preferred server starts and listens for state transfer (TCP/Redis/etc.) and QUIC packets (UDP:4433)
2. Primary server starts and listens for QUIC connections (UDP:4433). It knows the preferred server's address.

Phase 1: QUIC Handshake (Client <-> Primary Server)

3. Client connects to primary server at 141.217.168.152:4433
4. TLS 1.3 handshake happens. Transport parameters include: preferred_address = 141.217.168.143:4433
5. Handshake completes. Connection is "Connected".

Phase 2: State Export and Transfer (Primary -> Preferred)

6. Primary server detects handshake completion
7. Primary server calls export_migration_state() which captures: write/read crypto secrets, all connection IDs, client address, QUIC version, packet numbers
8. State is serialized to ~445 bytes
9. State is transferred to preferred server via the configured backend (TCP/HTTP/Redis)

Phase 3: State Import (Preferred Server)

10. Preferred server receives the state
11. Deserializes and reconstructs encryption keys via hkdf::import_key()
12. Derives AEAD key (payload encryption) and HP key (header protection) for both directions
13. Preferred server is now ready to decrypt client packets

Phase 4: Path Validation (Client <-> Preferred Server)

14. Client sends PATH_CHALLENGE to preferred address (141.217.168.143:4433)
 15. Preferred server receives, matches DCID, removes header protection, decrypts payload
 16. Finds PATH_CHALLENGE frame with 8-byte challenge data
 17. Builds PATH_RESPONSE with same 8 bytes, encrypts, sends back
 18. Client receives PATH_RESPONSE, verifies the 8-byte match
 19. Migration complete. All future traffic goes to preferred server.
-

7. The Key Challenge: Moving Encryption Keys

Why This Is Hard

The keys are derived through a chain:

```
TLS Handshake
  |
  v
Master Secret
  |
  v (HKDF-Expand-Label)
Traffic Secret <-- This is what we export
  |
  +--> AEAD Key   (derived deterministically)
  +--> AEAD IV    (derived deterministically)
  +--> HP Key     (derived deterministically)
```

The Solution: Export the Traffic Secret

In Neqo's original code, the `CryptoDxAppData` struct does not retain the traffic secret after deriving keys. We added a `secret: SymKey` field that retains it:

```
// EXPORT (on primary server):
let secret_bytes = self.secret.key_data()?.to_vec();

// IMPORT (on preferred server):
let secret = hkdf::import key(TLS_VERSION_1_3, &secret_bytes)?;
let aead = Aead::new(TLS_VERSION_1_3, cipher, &secret, label)?;
let hpkey = hp::Key::extract(TLS_VERSION_1_3, cipher, &secret, &hplabel)?;
```

The key derivation is **deterministic** – given the same traffic secret, cipher suite, and labels, you always get the same AEAD and HP keys.

8. Code Changes to Neqo

We modified 4 existing files and added 1 new file in `neqo-transport`, plus created the `migration-test` crate with 2 binaries and a transfer module.

Modified Files

neqo-transport/src/crypto.rs: - Added `secret: SymKey` field to `CryptoDxAppData` to retain current traffic secret - Added `export_for_migration()` method to extract all state - Added `new_from_migration()` to reconstruct from raw bytes - Added `dx()/dx_mut()` accessors for direct packet operations

neqo-transport/src/connection/mod.rs: - Added `export_migration_state()` to gather all connection state - Added `new_from_migration()` to reconstruct a `Connection`

neqo-transport/src/cid.rs: - Added `sequence_number()`, `iter()`, `local_cids()` accessors for CID enumeration

neqo-transport/src/lib.rs: - Made crypto module public - Registered migration_state module

New Files

neqo-transport/src/migration_state.rs: - MigrationState, CryptoStateExport, CidExport structs - encode()/decode() serialization - import_crypto_app_data() key reconstruction

migration-test/src/transfer/ (module): - mod.rs – StateSender/StateReceiver traits, backend selector - tcp_push.rs – Backend 1: Direct TCP - http_pull.rs – Backend 2: HTTP REST API - redis_kv.rs – Backend 3: Redis SET/GET - redis_pubsub.rs – Backend 4: Redis Pub/Sub - file_backend.rs – Backend 5: Shared filesystem (not used cross-machine)

migration-test/src/primary_server.rs: - HTTP/3 server using modified Neqo - Exports state via pluggable backend after handshake

migration-test/src/preferred_server.rs: - Receives state via pluggable backend - Manual QUIC packet processing (header protection, decryption, frame parsing) - Handles PATH_CHALLENGE and sends PATH_RESPONSE

9. The Migration State Format

The total migration state is approximately **445 bytes**:

Field	Size (bytes)	Description

Maqic "MIGR"	4	Format identifier
QUIC Version	4	e.g., Version1 or Version2
Write Crypto State:		
Secret	~34	TLS traffic secret (len-prefixed)
Next Secret	~34	For key updates (len-prefixed)
Cipher Suite	2	e.g., AES_128_GCM
Direction	1	1 = Write
Epoch	4	Key generation number
Used PN Start	8	Packet number tracking
Used PN End	8	Packet number tracking
Min PN	8	Minimum packet number
Read Crypto State:	~99	(same structure as write)
Local CIDs (typically 8):	~90	seqno + CID bytes each
Remote CIDs (typically 1):	~12	seqno + CID bytes
Client Address:		
Type + IP + Port	7	IPv4 address

Total	~445 bytes	

10. How the Preferred Server Handles Packets

The preferred server does manual QUIC packet processing:

Step 1: Receive and identify

- Check short header (bit 7 = 0)
- Extract DCID and verify it matches one of our local CIDs

Step 2: Remove header protection

- Find HP sample (16 bytes starting 4 bytes after packet number)
- Compute HP mask, unmask first byte and packet number

Step 3: Decrypt payload

- Use `AEAD.decrypt(packet_number, header, ciphertext)`
- Result: plaintext QUIC frames

Step 4: Parse frames

- Find `PATH_CHALLENGE` (type 0x1a) with 8-byte challenge data
- Also handles PING, ACK, STREAM frames

Step 5: Build and send `PATH_RESPONSE`

- Construct short-header packet with `PATH_RESPONSE` frame (type 0x1b)
 - Same 8-byte challenge data
 - Pad to 1200 bytes (RFC requirement)
 - Encrypt with AEAD, apply header protection
 - Send via UDP to client
-

11. When to Transfer State (Timing Strategies)

The **timing** of state transfer is an independent research dimension from the transfer mechanism.

Strategy 1: Immediate Transfer (Post-Handshake)

This is our current implementation. State is transferred immediately after the TLS handshake completes, before the client sends `PATH_CHALLENGE`.

- **Advantage:** Preferred server is ready before client's `PATH_CHALLENGE` arrives. Simple timing.
- **Disadvantage:** Every connection triggers a transfer, even if client never migrates. Wasted work.
- **Best for:** Simple deployments, low-latency networks.

Strategy 2: Lazy Transfer (On-Demand)

State is only transferred when the preferred server receives a packet it cannot decrypt (unknown CID triggers a state lookup from the primary server).

- **Advantage:** No wasted transfers. Works with multiple preferred servers.
- **Disadvantage:** Additional latency on first `PATH_CHALLENGE`. Preferred must buffer packets.
- **Best for:** Large-scale deployments, when not all clients migrate.

Strategy 3: Pre-emptive Transfer (Prediction-Based)

State (or key derivation material) is distributed before the client connects, based on predictions.

- **Advantage:** Fastest migration – preferred server is always ready.
- **Disadvantage:** Requires prediction. Wasteful if wrong. Security risk (keys exist longer on two machines).
- **Best for:** Planned maintenance, predictable traffic.

Strategy 4: Continuous Synchronization

State is transferred initially and continuously updated as keys rotate.

- **Advantage:** Preferred server always has latest keys. Handles key updates.
- **Disadvantage:** Highest bandwidth. Most complex. Keys in transit frequently.
- **Best for:** Long-lived connections, production with key rotation.

12. How to Transfer State (Mechanisms)

We implement 4 state transfer backends, selected via `STATE_TRANSFER` environment variable.

Backend 1: TCP Push (STATE_TRANSFER=tcp)

Primary server pushes state directly to preferred server over a plain TCP connection.

- **Pattern:** Direct push
- **Coupling:** Tight (primary must know preferred's address)
- **Latency:** ~661us (fastest)
- **Dependencies:** None

Backend 2: HTTP Pull (STATE_TRANSFER=http)

Primary server exposes an HTTP endpoint. Preferred server pulls state when needed.

- **Pattern:** On-demand pull
- **Coupling:** Medium (preferred must know primary's API)
- **Latency:** ~11.4ms
- **Dependencies:** None

Backend 3: Redis KV (STATE_TRANSFER=redis_kv)

Primary writes state to Redis with SET + TTL. Preferred reads with GET.

- **Pattern:** Shared storage (push + pull)
- **Coupling:** Loose (both only know Redis address)
- **Latency:** ~1.49ms
- **Dependencies:** Redis server
- **Multi-server:** Supports N:M via CID-based keys

Backend 4: Redis Pub/Sub (STATE_TRANSFER=redis_ps)

Primary publishes state to a Redis channel. Preferred subscribes and receives instantly.

- **Pattern:** Event-driven push
- **Coupling:** Loose (both only know Redis address)
- **Latency:** ~1.07ms
- **Dependencies:** Redis server
- **Multi-server:** Supports fan-out to multiple preferred servers

Configuration

```
# Select backend
STATE_TRANSFER=tcp|http|redis_kv|redis_ps

# TCP Push
STATE TCP ADDR=141.217.168.143:9999
STATE_TCP_PORT=9999

# HTTP Pull
STATE HTTP PORT=9998
STATE_HTTP_PRIMARY=141.217.168.152:9998

# Redis (both KV and Pub/Sub)
REDIS_URL=141.217.168.143:6379
REDIS_TTL=60

# Multi-server instance ID
INSTANCE_ID=server1
```

13. Combining When and How

The “when” and “how” dimensions are orthogonal. Any timing strategy can use any mechanism:

	TCP Push -----	HTTP Pull -----	Redis KV -----	Redis Pub/Sub -----
Immediate (post-handshake)	Current impl.	Tested	Tested	Tested
Lazy (on-demand)	Not ideal	Natural fit	Natural fit	Not ideal
Pre-emptive (prediction)	Possible	Possible	Natural fit	Natural fit
Continuous (sync)	Possible	Possible	Natural fit	Natural fit

Best combinations: - **Immediate + TCP Push** – Simplest, fastest. Good for demos. - **Lazy + Redis KV** – Scalable, decoupled. Good for production. - **Immediate + Redis Pub/Sub** – Fast, event-driven. Good for fan-out. - **Continuous + Redis KV** – Most robust. Good for long-lived connections.

14. Modular Implementation

Trait Interface

```
trait StateSender {
    fn send_state(&mut self, data: &[u8], instance_id: &str) ->
TransferResult<TransferMetrics>;
    fn name(&self) -> &str;
}

trait StateReceiver {
    fn receive_state(&mut self, instance_id: &str) -> TransferResult<(Vec<u8>,
TransferMetrics)>;
    fn name(&self) -> &str;
}
```

File Structure

```
migration-test/src/
  primary server.rs      -- Uses StateSender trait
  preferred_server.rs    -- Uses StateReceiver trait
  transfer/
    mod.rs               -- Trait definitions + backend selector
    tcp push.rs          -- Backend 1
    http pull.rs         -- Backend 2
    redis kv.rs          -- Backend 3
    redis pubsub.rs      -- Backend 4
    file_backend.rs      -- Backend 5 (not used cross-machine)
```

Timing Metrics

Every backend records timestamps: - send_start, send_end (on primary) - receive_start, receive_end (on preferred) - state_size (bytes)

15. Benchmark Results

The Combination Matrix (When x How)

We test every combination of timing strategy (WHEN) and transfer mechanism (HOW).

X-axis (columns): HOW to transfer – the 4 mechanisms **Y-axis (rows):** WHEN to transfer – the 2 timing strategies

	TCP Push	HTTP Pull	Redis KV	Redis Pub/Sub
IMMEDIATE (block, wait for state first)	PASS 595 us	PASS 2.5 ms	PASS 1.4 ms	PASS 1.1 ms
LAZY (listen for packets first, fetch)	FAIL (state lost if nobody	PASS 1.2 ms	PASS 983 us	FAIL (message lost if nobody

```
| on-demand) | listens) | | subscribed) |
+-----+
```

Why Some Combinations Fail

Lazy + TCP Push = FAIL: In lazy mode, the preferred server starts listening for QUIC packets first (not TCP). When the primary server tries to push state via TCP, nobody is listening on the TCP port yet. The TCP connection is refused and the state is lost.

Lazy + Redis Pub/Sub = FAIL: In lazy mode, the preferred server hasn't subscribed to the Redis channel yet when the primary publishes. Redis Pub/Sub delivers messages only to currently subscribed clients. If nobody is subscribed, the message is dropped.

Lazy + Redis KV = PASS: Redis KV stores the state persistently with SET. When the preferred server receives an unknown packet and looks up the CID in Redis, the state is still there waiting. This is why shared storage works best with lazy timing.

Lazy + HTTP Pull = PASS: The primary server keeps the state in memory and serves it via HTTP when the preferred server requests it. The preferred server polls until the HTTP endpoint is available.

What This Means

The combination matrix reveals a fundamental insight:

Lazy timing requires persistent state storage. If the state is ephemeral (TCP connection, pub/sub message), it can be lost if the preferred server isn't ready. If the state is persistent (Redis key, HTTP endpoint), the preferred server can fetch it whenever it needs to.

This means: - **For simple setups:** Immediate + TCP Push (fastest, simplest, 595us) - **For production:** Lazy + Redis KV (scalable, decoupled, 983us, state persists) - **Best overall:** Redis KV works with BOTH timing strategies and is fast in both

Detailed Latency Results (20 runs each, cross-machine)

Immediate timing (preferred waits for state, then listens):

Backend	Mean	Min	Max	P50
TCP Push	665 us	308 us	759 us	717 us
Redis Pub/Sub	1.3 ms	1.0 ms	1.6 ms	1.4 ms
Redis KV	1.3 ms	1.0 ms	1.6 ms	1.3 ms
HTTP Pull	42 ms	1.7 ms	102 ms	19 ms

Lazy timing (preferred listens first, fetches state on first packet):

Backend	Fetch Time	Status
Redis KV	983 us	PASS - fastest lazy option
HTTP Pull	1.2 ms	PASS - slightly slower
TCP Push	N/A	FAIL - state lost
Redis Pub/Sub	N/A	FAIL - message lost

Success Rate

All valid combinations achieved **100% success rate** across all test runs. Every client connected, every state was transferred, every PATH_CHALLENGE was decrypted, every PATH_RESPONSE was sent.

Security Analysis

Each backend handles TLS secrets differently during transfer:

Factor	TCP Push	HTTP Pull	Redis KV	Redis PS
Secrets in transit	Plaintext TCP	Plaintext (add TLS)	Plaintext to Redis	Plaintext broadcast
Exposure window	Brief send+close	Until pulled	Until TTL expires	Instant broadcast
At-rest storage	None	Primary memory	In Redis persists!	None
Who can intercept	Network sniffer	Network sniffer	Anyone with Redis access	All Redis sub-scribers
Can add encryption	Hard	Easy (HTTPS)	Redis AUTH+TLS	Redis AUTH+TLS
Multi-server risk	Low 1:1 direct	Low 1:1 pull	High any reader	High all get secrets

Security ranking: HTTP Pull > Redis KV > TCP Push > Redis Pub/Sub

HTTP Pull is most secure because secrets stay in primary server memory until explicitly requested by the specific preferred server that needs them.

Scalability Analysis

Scalability measures whether the system works with many servers without changing code or configuration.

TCP Push (LOW): Primary must open a direct TCP connection to the specific preferred server. If you have 100 preferred servers, the primary must know exactly which one to connect to. Adding servers requires reconfiguration.

HTTP Pull (MEDIUM): Preferred server pulls from primary, but must know which primary handled the client. Still requires some knowledge of the other side. Adding preferred servers is easy, but they must discover the right primary.

Redis KV (HIGH): Primary writes state with the CID as key. Any preferred server that receives a packet simply looks up the CID in Redis. No server needs to know about any other server – only Redis. Adding or removing servers requires zero configuration changes.

Redis Pub/Sub (MEDIUM-HIGH): Primary publishes, all preferred servers subscribe. Adding preferred servers is easy (just subscribe). But ALL subscribers receive ALL state, which is wasteful

and a security concern at scale.

Scenario	TCP Push	HTTP Pull	Redis KV	Redis PS
1 primary 1 preferred	Works	Works	Works	Works
3 primary 3 preferred	Must know which preferred to push	Must know which primary to pull	Just works (CID key)	Just works (but all get all)
100 primary 100 preferred	Does not scale	Complex routing	Just works	Wasteful broadcast

Reliability Analysis

Reliability measures what happens when things go wrong (server crash, network failure, restart).

Failure	TCP Push	HTTP Pull	Redis KV	Redis PS
Preferred server down when state is sent	TCP connect fails, state lost	No problem (pulled later)	State safe in Redis, retry anytime	Message lost forever
Primary server crashes after sending state	No effect (already sent)	State lost (was in memory)	State safe in Redis	No effect (already published)
Redis goes down	N/A	N/A	State lost	Pub/Sub broken
Network hiccup during transfer	Fails	Retries (polls)	Retries (polls)	May miss message

Reliability ranking: Redis KV > HTTP Pull > TCP Push > Redis Pub/Sub

Redis KV is most reliable because state persists independently of both servers. Even if both servers crash and restart, the state is still in Redis.

Overall Scoring (1-5, higher is better)

Metric	TCP Push	HTTP Pull	Redis KV	Redis PS
Latency	5	2	4	4
Security	1	4	3	1
Scalability	1	3	5	4
Reliability	2	3	5	2
Timing flexibility	2	5	5	2
Loose coupling	1	3	5	5

Simplicity	5	3	3	3
No dependencies	5	5	2	2
Persistence	1	2	5	1
-----+				
TOTAL	23	30	37	24
-----+				

Master Table: All 8 Combinations x All Metrics

This is the complete evaluation of every When x How combination across all research metrics.

Combination	Latency	Security	Scala-	Relia-	Coupling	Complex	Deps
Persist			bility	bility			
-----+							
IMMEDIATE							
+ TCP Push	734us	LOW	LOW	LOW	TIGHT	LOW	NONE
	BEST	plaintext	1:1	fails	must	simple	gone
		brief	only	if	know	TCP	after
		exposure		down	addr		send
-----+							
IMMEDIATE							
+ HTTP Pull	35ms	MED	MED	MED	MED	MED	NONE
	SLOW	can add	need	retry	must	HTTP	in
primary		HTTPS	to	polls	know	server	
			know		primary	+client	memory
			primary				
-----+							
IMMEDIATE							
+ Redis KV	1.2ms	MED	HIGH	HIGH	LOOSE	MED	REDIS
	GOOD	in Redis	N:M	survives	only	RESP	server
		delete	by CID	crash	know	proto	TTL
		after			Redis		
		read					
-----+							
IMMEDIATE							
+ Redis PS	1.5ms	LOW	MED-	LOW	LOOSE	MED	REDIS
							NONE

		G00D	secrets	HIGH	message	only	RESP	server	gone
			go to	fan-	lost	know	+sub		after
			ALL subs	out	if no	Redis			send
					sub				
+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
LAZY									
+ TCP Push	479us	MED	LOW	MED	TIGHT	MED	NONE	NONE	
	BEST	secrets	1:1	bg thrd	must	TCP			in
memory	import:	stay on	only	waits	know	+thrd			
	7us	primary			addr				brief
		longer							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
LAZY									
+ HTTP Pull	25ms	HIGH	MED	MED	MED	MED	NONE	MEM	
	SLOW	secrets	need	retry	must	HTTP			in
primary	import:	stay in	to	polls	know	+thrd			
	24ms	primary	know		primary				until
pulled		until	primary						
		pulled							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
LAZY									
+ Redis KV	1.5ms	MED	HIGH	HIGH	LOOSE	MED	REDIS	YES	
	G00D	in Redis	N:M	survives	only	RESP	server		until
	import:	delete	by CID	crash	know	+thrd			TTL
	3.8ms	after			Redis				
		read							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
LAZY									
+ Redis PS	1.5ms	LOW	MED-	MED	LOOSE	MED	REDIS	NONE	
	G00D	secrets	HIGH	bg thrd	only	RESP	server		in
memory	import:	go to	fan-	waits	know	+sub			
	425us	ALL subs	out		Redis	+thrd			brief

+-----+-----+-----+-----+-----+-----+-----+-----+
+

Scoring: All 8 Combinations (1-5 scale, higher is better)

Combination	Lat	Sec	Scl	Rel	Cpl	Sim	Dep	Per	TOTAL
Imm + TCP Push	5	1	1	2	1	5	5	1	21
Imm + HTTP Pull	2	3	3	3	3	3	5	2	24
Imm + Redis KV	4	3	5	5	5	3	2	5	32
Imm + Redis PS	4	1	4	2	5	3	2	1	22
Lazy + TCP Push	5	2	1	3	1	3	5	1	21
Lazy + HTTP Pull	2	5	3	3	3	3	5	3	27
Lazy + Redis KV	4	3	5	5	5	3	2	5	32
Lazy + Redis PS	4	1	4	3	5	3	2	1	23

Lat=Latency, Sec=Security, Scl=Scalability, Rel=Reliability,
Cpl=Coupling(loose=better), Sim=Simplicity, Dep=No Dependencies, Per=Persistence

Recommendations

Best overall (32/40): Immediate + Redis KV and Lazy + Redis KV are tied. Redis KV wins regardless of timing strategy because it scores highest in scalability, reliability, persistence, and coupling.

Best for security (27/40): Lazy + HTTP Pull. Secrets never leave primary server memory until explicitly requested. HTTPS can be added for encrypted transfer. No third-party storage.

Best for demos (21/40): Immediate + TCP Push. Fastest latency, simplest code, no dependencies. But worst in security, scalability, and coupling.

Best latency: Lazy + TCP Push (479us send + 7us import) and Immediate + TCP Push (734us). Direct TCP is always fastest, but at the cost of everything else.

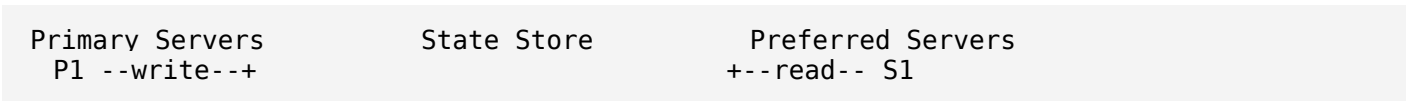
Best for production: Immediate + Redis KV or Lazy + Redis KV. High scalability (N:M routing by CID), high reliability (survives crashes), persistent state, fully decoupled. Only downside is requiring a Redis server.

Not recommended: Redis Pub/Sub. It requires the same Redis dependency as Redis KV but scores lower in security (broadcasts to all) and persistence (messages are ephemeral). Use Redis KV instead.

16. Multi-Server Migration (N:M)

The Concept

In a datacenter, you have N primary servers and M preferred servers. When a client connects to any primary, its state must reach the correct preferred server.



```
P2 --write--> Redis / DB <---read-- S2
P3 --write-- +---read-- S3
```

How Each Backend Handles N:M

- **TCP Push:** Primary must know which preferred to connect to. Tight coupling, poor scalability.
- **HTTP Pull:** Preferred must know which primary to pull from. Medium coupling.
- **Redis KV:** Any primary writes with CID-based key. Any preferred reads by CID. Fully decoupled.
- **Redis Pub/Sub:** Primary publishes with instance-based channel. Correct preferred subscribes. Decoupled.

Implementation

Each instance uses INSTANCE_ID environment variable for routing: - Redis keys: `quic_migration:<instance_id>` - Redis channels: `quic_migration:ch:<instance_id>` - TCP/HTTP: separate ports per instance

17. Security Implications

The QUIC-Exfil Attack

This cross-machine migration demonstrates the QUIC-Exfil attack:

1. Malicious server accepts QUIC connection from victim
2. Advertises preferred_address pointing to attacker's server
3. Transfers crypto state to attacker
4. Client migrates to attacker's server
5. Attacker receives all client's traffic

Why Firewalls Cannot Stop This

- The preferred_address is inside the **encrypted** handshake
- PATH_CHALLENGE/RESPONSE are **encrypted**
- Post-migration packets look identical to normal QUIC traffic
- ML-based classifiers achieve only **0-47% F1-score**

State Transfer Security

The migration state contains TLS traffic secrets in plaintext. Intercepting the transfer gives: - Ability to decrypt all current session packets - Ability to inject forged packets - Ability to impersonate either server

Recommendations: 1. Encrypt the transfer channel (TLS between servers) 2. Authenticate both endpoints (mutual TLS or shared HMAC) 3. Minimize exposure time (immediate transfer with short TTLs) 4. Audit all transfers (logging for security monitoring)

Theoretical Alternative: Pre-Shared Key

Instead of transferring secrets, both servers share a master secret at deployment and derive session keys independently. This means secrets never traverse the network. However, this breaks

TLS forward secrecy and requires deep modifications to the TLS implementation. Discussed as future work.

18. Running the Demo

Prerequisites

- Rust toolchain (rustup)
- clang + libclang-dev (for NSS compilation)
- Redis server on a dedicated machine (141.217.168.200, for redis_kv/redis_ps backends)
- Four machines on the same network: client, primary server, preferred server, Redis server

Quick Start with Aliases

On homeserver2 (preferred):

```
preferred-up      # Start preferred server (TCP backend)
preferred-down    # Stop
```

On opti7040 (primary), two terminals:

```
bootstrap-up      # Terminal 1: TCP HTTPS for Firefox
primary-up        # Terminal 2: QUIC primary server
primary-down      # Stop
```

Firefox: navigate to <https://141.217.168.152:4433/>

Switching Backends

```
# On preferred server:
STATE_TRANSFER=redis_kv REDIS_URL=127.0.0.1:6379 preferred-up

# On primary server:
STATE_TRANSFER=redis_kv REDIS_URL=141.217.168.143:6379 primary-up
```

Running Benchmarks

```
# Experiment 1: 20 runs per backend
./benchmarks/run_benchmark.sh 20

# Experiment 2: Multi-server (3 primary, 3 preferred)
./benchmarks/run_multiserver.sh

# Experiment 3: Concurrent clients
./benchmarks/run_concurrent.sh 5
```

Firefox Configuration

```
Set in about:config: - network.http.http3.enabled = true - network.http.http3.alt-svc-  
mapping-for-testing = 141.217.168.152:4433;h3=":4433" -  
network.http.http3.disable_when_third_party_roots_found = false
```

Import server certificate: Settings > Privacy & Security > Certificates > View Certificates > Authorities > Import > /tmp/quic_cert.pem

What You Should See

Primary server:

```
=== PRIMARY SERVER (HTTP/3) ===  
[Transfer] Backend: tcp  
Connection state: Connected  
>>> EXPORTING MIGRATION STATE <<<  
State size: 445 bytes  
State sent successfully!  
[Metrics] backend=tcp, send_time=661us
```

Preferred server:

```
=== PREFERRED SERVER ===  
[Transfer] Backend: tcp  
State received (445 bytes)  
Crypto imported!  
DCID 75d73c9f8b9efef5 MATCH  
DECRYPTED! 1226 bytes plaintext  
Frame: PATH CHALLENGE data=dbb1dce57902c95b  
>>> SENDING PATH RESPONSE <<<  
>> Sent PATH_RESPONSE (1200 bytes)
```

19. Future Work

1. **Lazy (on-demand) transfer** – Restructure preferred server to fetch state when unknown CID arrives, rather than blocking on state receipt at startup
2. **Continuous synchronization** – Keep preferred server's keys updated as TLS key rotation happens on the primary
3. **Pre-shared key derivation** – Explore approaches where both servers derive keys from shared master secret, eliminating secret transfer entirely
4. **Encrypted state transfer** – Add TLS to the inter-server channel to protect secrets in transit
5. **Larger-scale benchmarks** – Test with 10+ primary/preferred servers and hundreds of concurrent clients
6. **Docker-based deployment** – Containerize for easy reproduction without physical machines
7. **Wireshark integration** – Automated packet capture and analysis during benchmarks

20. Glossary

- **AEAD** – Authenticated Encryption with Associated Data. Encrypts data and verifies integrity.
- **CID** – Connection ID. Random identifier used to match packets to connections.
- **DCID** – Destination Connection ID. The CID in the packet header.
- **Epoch** – Key generation number. Increments with each key update.

- **HKDF** – HMAC-based Key Derivation Function. Derives keys from secrets.
- **HP** – Header Protection. Masks packet numbers so observers can't track them.
- **Neqo** – Mozilla's QUIC implementation in Rust.
- **NSS** – Network Security Services. Mozilla's crypto library used by Firefox and Neqo.
- **PATH_CHALLENGE** – QUIC frame sent to validate a new network path.
- **PATH_RESPONSE** – QUIC frame echoing the challenge data to prove path ownership.
- **Preferred Address** – A QUIC transport parameter where the server suggests a new address.
- **Short Header** – Compact QUIC packet format used after the handshake.
- **SymKey** – NSS type representing a symmetric cryptographic key.
- **TLS 1.3** – The cryptographic handshake protocol used by QUIC.
- **Transport Parameter** – Configuration values exchanged during the QUIC handshake.